RSnowflake in Custom SPCS Services

This vignette covers **RSnowflake** in **custom Snowpark Container Services (SPCS)** workloads — for example **RStudio Server** deployed as a long-running service. It is **not** the Workspace Notebook path (`%%R` via rpy2); see `vignette("workspace-rsnowflake", package = "RSnowflake")` for that.

For the full deploy kit (Docker, service spec, smoke tests), see the **snowflakeR** package:

```
system.file("rstudio-spcs", package = "snowflakeR")
```

Or the Hitchhiker's Guide chapter **RStudio Server on SPCS**.

Execution model

In a custom SPCS **service** container:

1. Snowflake injects `SNOWFLAKE_HOST` and `/snowflake/session/token`.
2. R runs as a **native process** (not via rpy2).
3. `dbConnect(Snowflake())` uses the internal gateway + OAuth token.

There is no Python bootstrap cell. Set warehouse, database, schema, and role via **container environment variables** in the service spec.

Connect

```
library(DBI)
library(RSnowflake)

con <- dbConnect(Snowflake())
dbGetQuery(con, "SELECT CURRENT_USER(), CURRENT_WAREHOUSE()")
```

RSnowflake resolves auth in this order (`RSnowflake/R/auth.R`):

1. Explicit bearer `token` argument
2. **SPCS OAuth** — `SNOWFLAKE_HOST` + token file
3. `SNOWFLAKE_PAT`
4. Key-pair JWT

Warehouse and context

Custom SPCS services may **not** export `SNOWFLAKE_WAREHOUSE` automatically. Set it in the service YAML:

```
env:
  SNOWFLAKE_WAREHOUSE: ANALYTICS_WH
  SNOWFLAKE_DATABASE: MY_DB
  SNOWFLAKE_SCHEMA: MY_SCHEMA
  SNOWFLAKE_ROLE: MY_ROLE
```

RSnowflake passes these to the SQL API session. **Do not** rely on:

```
dbExecute(con, "USE WAREHOUSE MY_WH")   # not supported - SQL REST API
```

The SQL REST API does not support `USE`, `USE WAREHOUSE`, `GET`, or `PUT`. Use env vars or fully qualified SQL instead.

Verify token injection

Run once after the service starts:

```
cat("SNOWFLAKE_HOST:", Sys.getenv("SNOWFLAKE_HOST"), "\n")
cat("Token file:", file.exists("/snowflake/session/token"), "\n")
```

If the token file is missing, check your SPCS service spec and platform version, or fall back to PAT/key-pair against the public endpoint (with EAI).

RStudio Connections pane

RSnowflake registers with the RStudio Connections pane. Probe messages tagged `[RSnowflake pane]` are normal when the pane checks driver availability. To show verbose pane diagnostics:

```
options(rsnowflake.connections_pane_verbose = TRUE)
```

Stage file I/O

For bulk file transfer in custom services, mount a stage as a volume:

```
volumeMounts:
- name: stage-vol
  mountPath: /data/stage
volumes:
- name: stage-vol
  source: "@MY_DB.MY_SCHEMA.MY_STAGE"
```

Read and write via the filesystem path in R. Do not use `GET`/`PUT` SQL with RSnowflake.

Optional ADBC

When `adbcsnowflake` is baked into the image, RSnowflake can use Arrow-native bulk paths via the same internal gateway. See `WORKSPACE_ADBC.md` in the RSnowflake repository for gateway and redirect-chain detail (applies to SPCS containers as well as Workspace).

Companion: snowflakeR

ML platform APIs (`sfr_connect()`, Feature Store, Model Registry) require Snowpark Python via **reticulate**. In custom SPCS services, **`sfr_connect()` does not auto-detect** the environment — use `sfr_connect_spcs()` from the rstudio-spcs kit or `vignette("rstudio-spcs", package = "snowflakeR")`.